OPEN SOURCE TOOLS FOR DATA SCIENCE

LicenseLens

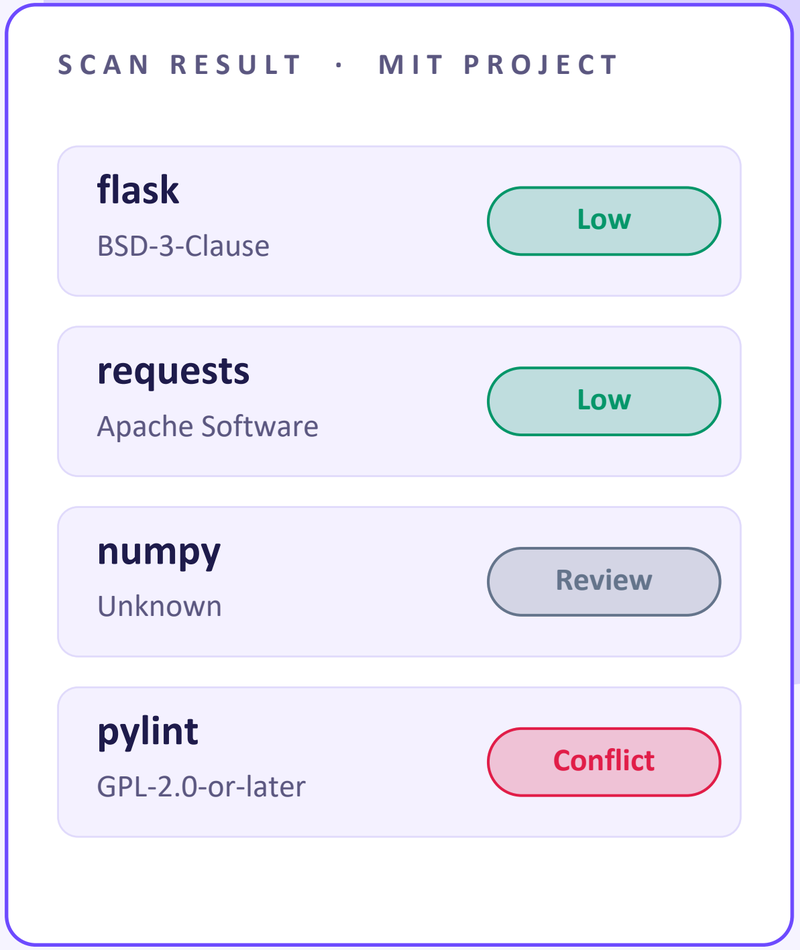
Open-source dependency & license compliance platform with SBOM generation, built as a containerised multi-service app

Sanobar Shaikh

PRN: 25030421039

Prajakta Suryavanshi

PRN: 25030421029



Symbiosis Institute of AI · BSc AI · Batch 2026

WHY IT MATTERS

The Problem

Every modern project depends on dozens of open-source packages. Few teams check their licenses.



Hidden legal risk

A GPL library inside an MIT or proprietary project can force the whole project to be open-sourced.



Manual checking fails

Each package has its own license and nested dependencies. Reading them by hand does not scale.



No proof of compliance

Customers and auditors now ask for a Software Bill of Materials (SBOM). Most small teams have none.

PROBLEM STATEMENT

Open-source projects often use third-party dependencies without checking license compatibility, creating legal and compliance risk (for example, a GPL library inside an MIT project).

WHAT WE BUILT

Our Solution: LicenseLens

Upload a dependency file. Get licenses, risks and an SBOM in seconds.

What it does

- Reads requirements.txt
- Fetches each package license from PyPI
- Flags risky and conflicting licenses
- Generates a CycloneDX SBOM + CSV report

Objectives

- Detect license conflicts automatically
- Show results in a clean dashboard
- Run as separate containers via Docker Compose
- Demonstrate open-source licensing concepts

Dataset

PyPI Package License Metadata
pypi.org/pypi/<package>/json (public API)
License reference: SPDX License List

Expected outcome

A small, reproducible tool. Target: one command (docker compose up) starts the dashboard, API (with the scanner module) and database.

WORKFLOW

How the App Works

End-to-end workflow, from upload to compliance report

1
Upload
requirements.txt

→

2
FastAPI
/scan endpoint

→

3
PyPI JSON API
license lookup

→

4
Risk engine
High / Med / Low

→

5
Dashboard
+ SBOM export

Risk rules

High	GPL / AGPL
Medium	LGPL / MPL / EPL
Low	MIT, BSD, Apache
Conflict	High risk inside a non-copyleft project

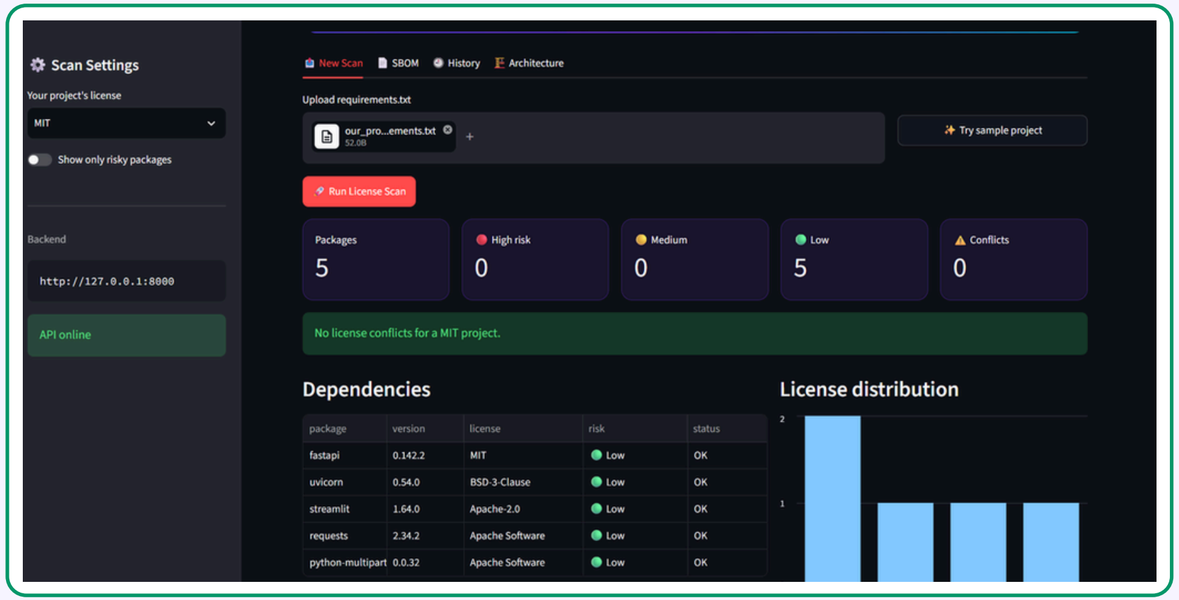
Outputs

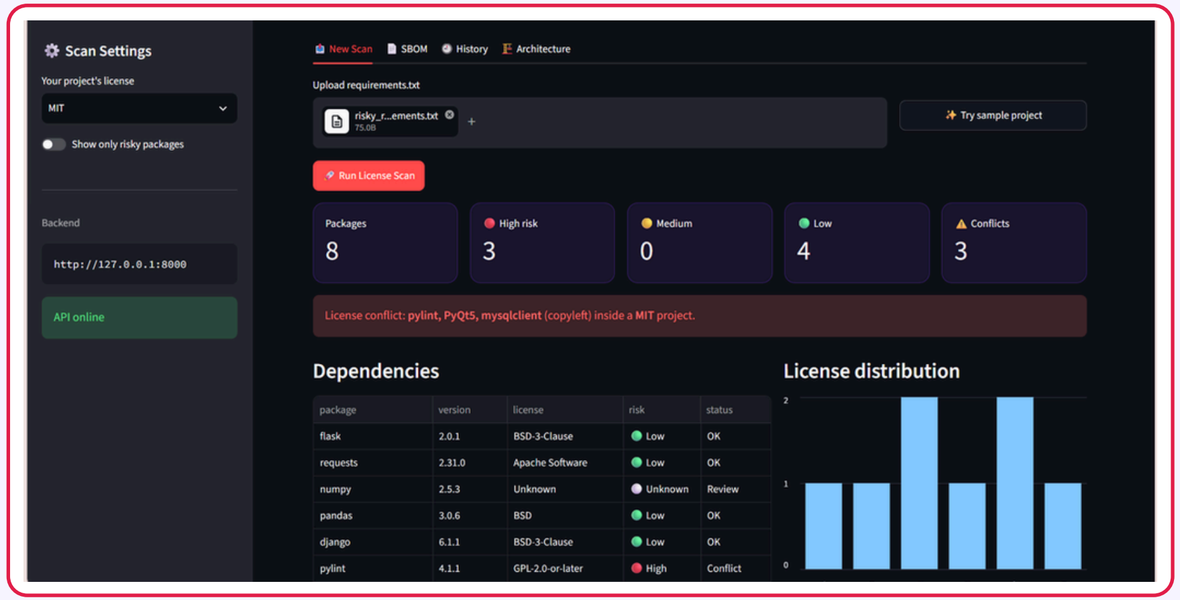
- Metrics: packages, high, medium, low, conflicts
- Dependency table and license chart
- SBOM in CycloneDX JSON
- CSV compliance report

LIVE

Working Demo

Live Streamlit dashboard talking to the FastAPI backend





Our own project: 5 packages, 0 conflicts

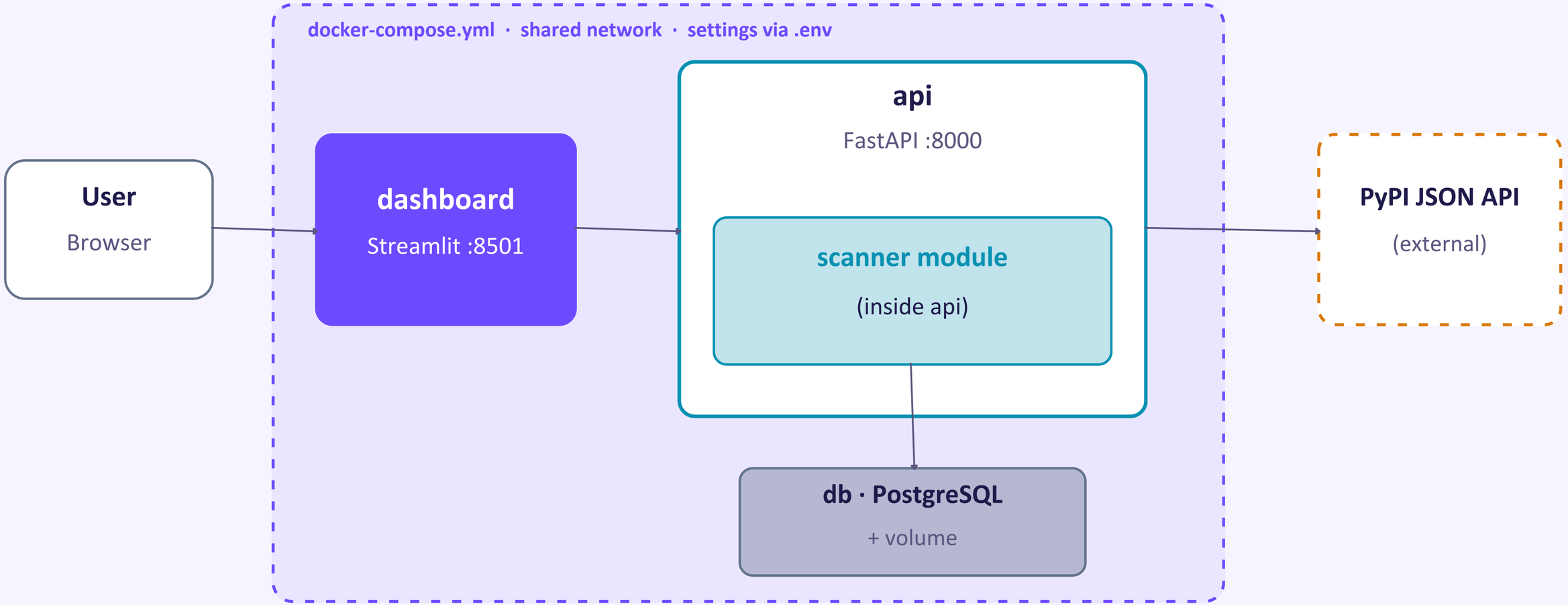
All dependencies use permissive licenses (MIT, BSD, Apache-2.0), so the project is safe to ship under MIT.

Risky project: 3 conflicts flagged

pylint, PyQt5 and mysqlclient are GPL. The tool raises a conflict for an MIT project. Switching the project to GPL-3.0 clears it.

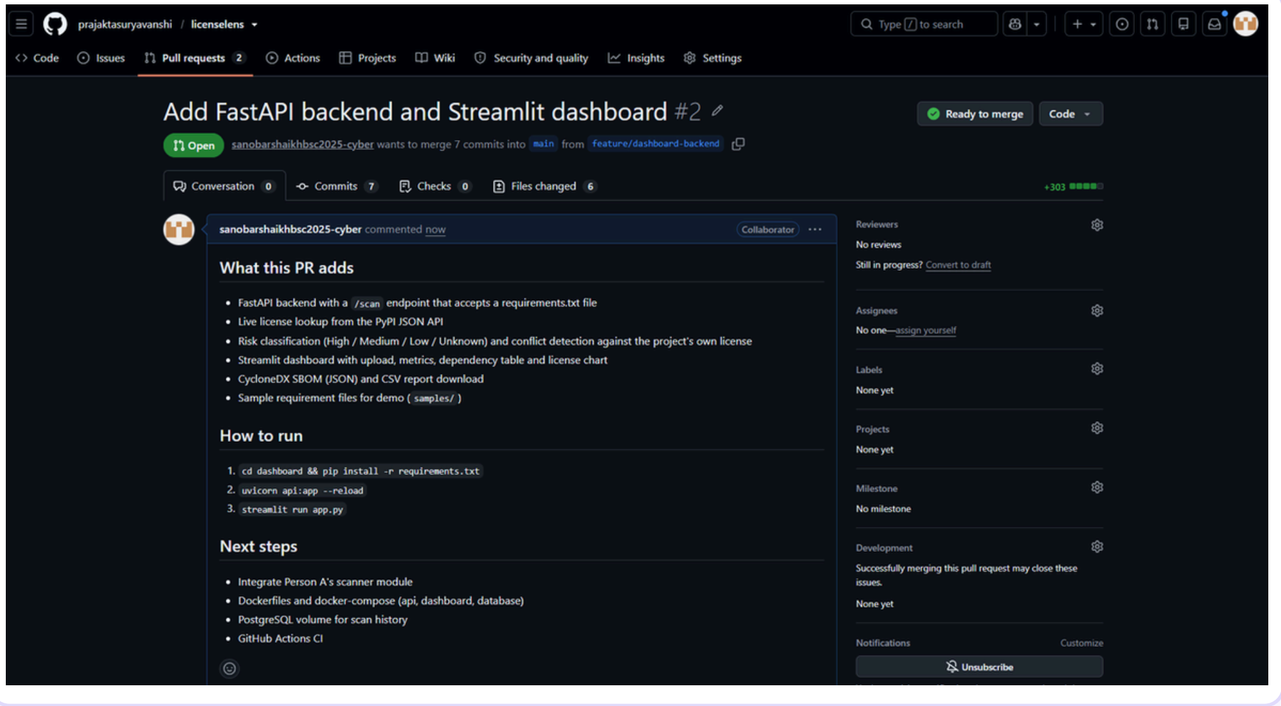
Containerisation & Compose Architecture

Three containers on one Docker Compose network: dashboard, api (scanner runs inside it) and db



GitHub Workflow & Commit Discipline

Feature branches, small meaningful commits, three pull requests and code review



Pull request #2: Add FastAPI backend and Streamlit dashboard

Example commits: feat: add FastAPI /scan endpoint · feat: add PyPI license lookup · fix: handle long license strings

#1 **Scanner module**

Dependency scanner added as an api module

#2 **Backend + dashboard**

/scan endpoint, risk engine, SBOM + CSV

#3 **Caching, tests and CI**

lru_cache, unit tests, GitHub Actions CI

What we followed

- Small commits: feat / fix / docs / chore
- Partner review comments, replies posted
- .gitignore keeps venv and .env out

STACK

Tools Used

Open-source tools across the whole workflow



Git + GitHub

Branches, commits, pull requests, reviews



FastAPI + Uvicorn

Backend API and /docs



Streamlit

Dashboard prototype



Docker + Compose

Multi-service containers



.env + volumes

Credentials and data persistence



GitHub Actions

CI: tests run on every push

LICENSING CONCEPTS APPLIED

MIT · Apache-2.0 · BSD · GPL · LGPL · SPDX identifiers · SBOM (CycloneDX)



Thank you

Questions & Discussion

B.Sc. Artificial Intelligence | Symbiosis Institute of Artificial Intelligence